Project Concept

TID

The Problem & The Solution — Conceptual Overview

Author	Ahmad Qasim Mohammad Hassan
ORCID	0009-0001-4360-0802
Version	1.0

First What Is the Problem?

When any program finishes working with sensitive data (passwords, encryption keys, session tokens, payment card data), that data remains in place inside RAM and cache memory until overwritten by other data. This gap creates three compounding risks:

1) Data persistence in main memory (RAM):
Even after memory is released with free, the content remains in the pages until they are rewritten or reallocated to another process.

2) Data persistence in cache layers (L1/L2/L3):
Even when the location in main memory is wiped, a copy may remain in the cache that authorised code can read later.

3) Compiler interference (Compiler optimisation):
Modern compilers consider writing zeros to memory that is 'about to be freed' a pointless operation and delete it during optimisation. Result: the developer writes memset(buf, 0, n) before free(buf), but the generated code does not actually execute it.

Second Why Are Current Solutions Insufficient?

- memset is removable by the optimiser.
- memset_explicit (C23) and SecureZeroMemory on Windows do not cover the cache — they only address main memory.
- Disk encryption (LUKS / BitLocker) does not protect memory at runtime.
- swap and hibernation operations may copy sensitive memory to disk.
- Any solution operating solely in user space remains vulnerable to compiler interference and privileged kernel access.

Third What Is the TID Solution?

TID (The Instant Destroyer) is a Linux kernel module designed to deliver a single, precise, and definitive guarantee:

> **Core Guarantee:** Whenever it is invoked for a memory region, the data exits both memory and cache simultaneously, instantly, and the compiler cannot prevent it.

TID achieves this guarantee through three integrated engineering pillars:

1) Compiler-resistant wiping:
Using a processor instruction set instruction written in a way that the optimiser is incapable of removing or reordering.

2) Cache eviction for the target range:
Expelling all cache lines associated with the region from L1/L2/L3 back to main memory, ensuring no copy remains at any level.

3) Memory barrier:
Guaranteeing that all eviction operations complete before any subsequent instruction executes, and preventing speculative reordering by the processor.

Positioning inside the kernel — not in user space — grants TID:

- Direct, privileged access to memory and cache management.
- Immunity from the permission constraints that hobble user-space tools.
- A unified interface (ioctl) available to any authorised application.

Fourth What Does TID Deliver to the End User?

- A single call for safe and complete wiping of any memory region.

- Execution in nanoseconds for small sizes (372 ns measured), and a few microseconds for megabyte-sized regions (theoretical estimate — not yet measured).
- Works without modifications to the compiler, existing application, or additional hardware.
- Simple requirements: an x86_64 processor supporting CLFLUSHOPT, and Linux kernel 5.6 or newer.

Fifth Who Is TID Designed For?

- Cryptographic libraries that require a hard guarantee for key wiping.
- Payment systems and card data storage.
- Multi-tenant cloud environments where memory is reused across different workloads.
- Medical systems holding sensitive patient data.
- Any application handling short-lived secrets.

Sixth Project Limits — What TID Does NOT Claim

- Does not block side-channel timing attacks in terms of the measurement act itself — however, TID v2.0 defeats Flush+Reload attacks by evicting the cache before the attacker reaches it: the attacker measures and finds the cache empty and data zeroed (3.7x ratio).
- Within the documented research literature, TID is the first software solution to combine main memory wiping and cache eviction together at kernel level, without requiring hardware modification or compiler changes (compared to: libsodium, OpenSSL, memzero_explicit, SecureZeroMemory — none of them covers the cache at kernel level).
- Does not protect against cold-boot replay attacks if power is cut before wiping completes.
- Does not replace disk encryption for protecting data at rest.
- Performance and correctness are measured on x86_64 architecture only; other architectures are outside the scope of this release.

Seventh Reviewed Solutions & Comparison Results

The following solutions were surveyed to verify the absence of any precedent combining main memory wiping and cache eviction together at kernel level:

#	Library / Solution	Source	Result
[1]	libsodium — sodium_memzero()	libsodium.gitbook.io/doc/memory_management	Wipes main memory only — does not touch the cache
[2]	OpenSSL — OPENSSL_cleanse()	openssl.org/docs/man3.0/man3/OPENSSL_cleanse.html	Wipes main memory only — does not touch the cache
[3]	GNU glibc — explicit_bzero()	man7.org/linux/man-pages/man3/explicit_bzero.3.html	Wipes main memory only — does not touch the cache
[4]	C23 Standard — memset_explicit()	ISO/IEC 9899:2023 — Section 7.24.6.1	Wipes main memory only — does not touch the cache
[5]	Windows — SecureZeroMemory()	learn.microsoft.com — legacy/windows/desktop	Wipes main memory only — does not touch the cache
[6]	Linux Kernel — memzero_explicit()	linux/string.h — kernel.org	Wipes main memory only — does not touch the cache
[7]	TRESOR (Müller et al., USENIX 2011)	usenix.org/legacy/events/sec11/tech/full_papers/Muller.pdf	Protects keys inside processor registers — does not combine wiping with cache eviction
[8]	Flush+Reload (Yarom & Falkner, 2014)	usenix.org — usenixsecurity14	Describes the attack TID defends against — provides no defence
[9]	CLFLUSHOPT — Intel SDM Vol. 2A	Intel 64 and IA-32 Architectures SDM	The instruction used by TID — not employed in combination with memory wiping in any of the solutions above

Comparison Summary: Within the surveyed literature above, there is not a single software solution that combines compiler-resistant memory wiping and cache eviction at kernel level in a single synchronous call.

— End of Concept Document —